WHITE PAPER

Spine and Leaf

A Product Delivery Blueprint for the AI Era

Customer empathy is the foundation. Technology depth is the new ceiling.

A methodology for the Product Manager who builds, not routes.

EPIGRAPH

> *One day, AGI will do everything. Until that day arrives, we have to raise our baseline based on the ecosystem we have. Using the ecosystem without raising your baseline is a recipe for failure.*

— this paper, in one sentence

Abstract

The Product Manager role was forged in an era of expensive iteration, where producing a high-fidelity artifact required a team and a calendar. That era is over. Generative tools have collapsed the cost of producing working code, interactive demos, and architecture diagrams to near-zero — and yet most product organizations still operate on the old grammar of PRD, wireframe, ticket, build, demo. The cost of that mismatch is paid in translation loss: every hop between functions drops fidelity, and by the time the product ships, the original story has been re-interpreted four times.

This paper proposes Spine and Leaf as a methodology for the AI era. The Spine is the immutable core narrative — the problem, the user, the success metrics, the scope — written from a foundation of customer empathy. The Leaves are high-resolution artifacts the Product Manager produces directly: a working coded prototype for engineering, an interactive guided tour for go-to-market, an architecture diagram with explicit Build versus Leverage decisions, an interaction-state library for design, a traceability matrix for compliance. Each Leaf eliminates translation loss for a specific stakeholder. Together they compress the alignment phase that used to dominate the first half of every sprint.

The argument is not that PMs become engineers. The argument is that the cost curves have moved, and the PMs who use the new tools to externalize their model of the product directly — in the medium each stakeholder actually consumes — will deliver products with less translation loss and more customer truth in them. Customer empathy remains the floor; nothing else matters without it. Technology depth is the new ceiling, and it is the difference between PMs who lead AI delivery and PMs who watch it happen.

The paper's goal is to ask, plainly, what the best outcome a Product Manager can bring to the table looks like now that the cost curves have moved. Running yesterday's process with today's tools — using a generative assistant to write a faster PRD — leaves the translation chain intact and the methodology unchanged. The Spine-and-Leaf model proposes a different answer: a single source under version control, the Leaves expressed as code so that a change in one place propagates to every team's resolution, and a discipline of asking what new artifact the tools make possible rather than what old artifact they can produce faster. The new tools are delivering exactly what we ask of them. The methodology is what we choose to ask.

Contents

1. The Rediscovery

Product management was shaped by an earlier era's constraints. Producing a coded prototype required engineering time. Producing a click-through demo required a designer. Producing an architecture diagram required pulling an architect into a meeting. Each artifact was expensive — so the role evolved around the one cheap artifact: the document. The PRD. The strategy memo. The roadmap.

Documents had a brutal failure mode — translation loss at every handoff — but the alternative was more expensive. The discipline accepted the loss and built an entire vocabulary to mitigate it: stakeholder maps, RACI charts, alignment meetings, design reviews, demo days. Every one of those rituals exists to compensate for prose being the wrong medium for almost every stakeholder consuming it.

The constraint has lifted. Generative tools have collapsed the cost of producing high-fidelity artifacts. A coded prototype that once required a frontend engineer for a week is within reach of a PM in an afternoon. An architecture diagram can be drafted by the PM before the architect sees it. An interactive demo can be authored alongside the strategy memo that proposes it.

And yet most product organizations have not changed their grammar.

The clearest example of the failure mode is the team that uses an AI assistant to write a faster PRD. The PRD is unchanged — still prose, still consumed primarily by the PM and the lead designer, still filtered through every translation hop downstream. What changed is that the PM produces it in two hours instead of two days. The translation chain that consumes the next eight weeks is intact. The methodology did not move. Running yesterday's process with today's tools is the most expensive way to look productive.

This paper asks the better question: what is the best outcome a Product Manager can bring to the table now that the cost curves have moved? The old answer was the document. The new answer needs to be different, because the constraint that produced the document has lifted.

Translation loss is also time loss

The cost of the old grammar is not only fidelity. It is time. Mark McDonald's keynote at Gartner CPO 2026 named the cycle: the Intelligence Supercycle — a period of durable growth driven by enterprise AI adoption — with the explicit caveat that "how we built IT products will not be how we build AI businesses." The window between idea and ship has compressed. Teams that produce all stakeholder artifacts from a single source ship inside that window. Teams still translating prose to wireframe to ticket to code to demo are running yesterday's clock. Field cases report 10× faster artifact creation, 33% lifts in deal velocity, and 28% reductions in customer acquisition cost when teams compress the artifact chain.

Translation loss is the meme; time loss is the bill. Every translation hop is a calendar week the team does not get back. The Spine and Leaf model is, at its core, a way to refuse those weeks.

We are not learning a new role. We are rediscovering the role we always wanted, now that the tools are finally caught up.

2. The Paradigm Shift: From Router to Builder

The old model: the PM as human router

In the traditional operating model, the Product Manager is a router. Information arrives from customers, executives, sales, support, and the market. It is filtered, prioritized, and translated into a document. The document is handed to design, which translates it into wireframes. The wireframes are handed to engineering, which translates them into code. The code is handed to QA, which translates it into test cases. The build is handed to sales, which translates it into a demo.

Each translation hop drops fidelity. Some of the loss is unavoidable: the user's frustration, expressed in a customer interview, cannot be perfectly reproduced in a Figma frame. But most of the loss is structural — it exists because the medium of one stage is the wrong medium for the next. Prose is not actionable for an engineer. Wireframes are not testable for QA. A test plan is not exciting for a customer. The router model accepts this loss as the price of coordination, and asks the PM to spend most of their time mitigating it.

The cost of the router model is not the documents themselves. The cost is the calendar. By the time a PRD has been read, debated, wireframed, scoped, ticketed, built, and demoed, the original story has been interpreted four times by four functions, each of which is optimizing for a slightly different objective. The result is shippable, but it is not the product the spine described. It is a product that survived the chain.

The new model: the PM produces zero-translation artifacts

In the proposed model, the Product Manager produces the artifacts each stakeholder actually consumes, in the medium that stakeholder actually uses. Engineers consume DOM and code. Sales consumes click-through demos. Architects consume system diagrams. Designers consume interaction states. QA consumes traceability matrices. The PM produces the high-resolution version of the same idea in each of these formats, and circulates them in parallel.

This is not a claim that PMs become engineers, designers, or architects. It is a claim that the PM is now the producer of the artifacts that used to require those functions to produce. The function still owns the final implementation. But the alignment phase — the part that consumes the first half of every sprint — is paid before the sprint begins, by the PM, in artifacts that aren't

documents. As a Product Manager in this era it is no longer enough to define the problem. The PM produces artifacts that speed up each discipline by sharing the same story across functions at the right resolution for each. The PM does the initial hard work to keep the Spine — to shift left and establish a single source of truth — so the rest of the team can move in parallel rather than serially.

And on the question that comes up in every workshop on this material: this is not a replacement for PM intuition. The traditional PM lives in the world of intuition — sensing what users want, what the team can ship, what the market will pay for. That world is not gone. It is amplified. The new tools do not replace the PM's intuition; they let the PM externalize it directly, in artifacts each stakeholder can interrogate. Intuition that used to live in a strategy memo now lives in a coded prototype, a tour, a diagram, a matrix. The PM is not relegated to a less-intuitive role; the PM is making intuition more visible to the team than ever before. That is amplification, not replacement.

The result is a step-change in the latency from idea to aligned team. The PRD that used to take a week to read and a week to debate is replaced by a coded prototype the team can interact with in a meeting, a tour the executive can click through over coffee, and a diagram the architect can mark up in real time. The story is the same in all three; the resolution is different. Each stakeholder receives the version of the story they can actually use.

This is what we mean by zero-translation artifacts. They are not zero-effort — producing them takes work. They are zero-translation in the sense that the engineer does not have to translate prose into code, because the code is already there. The salesperson does not have to translate strategy into pitch, because the tour is already the pitch. The architect does not have to translate intent into a diagram, because the diagram is the proposal.

Why this works now and didn't before

The mechanics of the new model would have been impossible five years ago. Producing a credible coded prototype required a frontend engineer. Producing a polished tour required a designer. Producing an architecture diagram with depth required an architect. The PM could not have produced any of these in a reasonable timeframe.

What changed is not that PMs became more capable in the abstract. What changed is that the tools available to a PM became dramatically more powerful. Generative coding assistants compress the time-to-prototype by an order of magnitude. Diagram-generation tools turn a verbal description into a structured visual in seconds. Interactive demo frameworks let a PM author state changes the way they used to author bullet points. The marginal cost of producing a high-fidelity artifact is no longer a function of the engineering team; it is a function of the PM's clarity of thought.

And clarity of thought is what the PM was always supposed to bring. The role has not changed. The constraints around it have changed. The methodology in this paper is what the role looks like when the constraints lift.

3. The Role Shift in RACI Form

The paradigm shift in Section 2 is the argument. The RACI in this section is the argument's operational form — the same shift, expressed in the language every senior product organization already uses. RACI tells you who is Responsible, who is Accountable, who is Consulted, and who is Informed for each artifact. It is the question every team asks when a new methodology lands: but who actually does what?

The before-and-after comparison is the load-bearing claim. In the old model the PM is Accountable on one artifact — the PRD — and Informed on most others. The PM coordinates handoffs and inherits the translation loss. In the Spine and Leaf model the PM is Accountable on every artifact, and each discipline is Responsible for refining the Leaf they consume. The PM has not taken on more meetings; the PM has taken on more authoring. The conversations the PM used to broker between functions now happen in front of artifacts the PM has already authored. The role is not bigger in hours; it is denser in leverage.

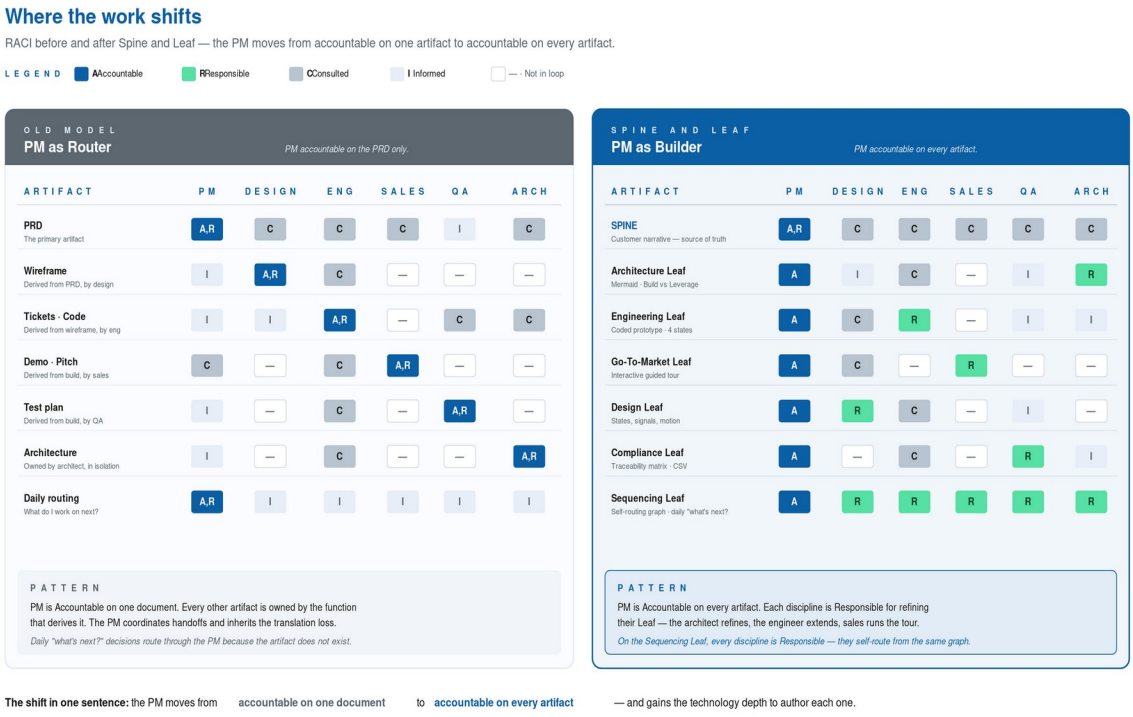

Figure 2. The PM moves from accountable on one artifact to accountable on every artifact. Each discipline is Responsible for refining their Leaf.

Reading the chart

Three patterns are worth naming because they will come up the first time a senior PM or CPO sees the methodology proposed:

- **The PM column compresses.** In the old model, the PM is Accountable on the PRD and on daily routing — the rest is Informed or Consulted. In the new model, the PM is Accountable on the Spine and every Leaf. This is the central operational claim. If the PM does not take ownership of authoring at this level, the methodology does not work; if the PM does, the team's velocity transforms.
- **Each discipline becomes Responsible on exactly one Leaf.** The architect refines the Architecture Leaf. The engineer extends the Engineering Leaf. Sales runs the GTM Leaf. The designer refines the Design Leaf. QA verifies the Compliance Leaf. The Leaves don't take expertise away — they direct it at a halfway draft, which is where the discipline's expertise has the highest leverage.
- **The Sequencing Leaf inverts the routing pattern.** On every other Leaf, one discipline is Responsible. On the Sequencing Leaf, every discipline is Responsible — they self-route from the same graph. This is the row that eliminates the daily 'what should I work on next?' Slack message. The PM authors the graph; the team routes from it.

The translation of this RACI into practice is straightforward. The PM blocks calendar time at the start of each cycle to author the Spine and the first drafts of the Leaves. Disciplines block calendar time to refine their Leaf rather than to interpret a PRD. The standing alignment meeting that used to dominate the first half of every sprint is replaced by a single review session in front of the artifacts. The result is a flatter, faster operating model — and the RACI is what makes the flattening legible to a leadership team that has to fund it.

4. The Spine: The Immutable Core Narrative

The Spine is the single document of record for a product initiative. It is the immutable referent — the place every other artifact returns to, the place every disagreement gets resolved, the place every new team member starts. When in doubt, return to the Spine. If the Spine doesn't say it, the artifact has no authority to say it either.

The Spine is short. It is plain language. It is in the user's voice wherever possible. It is not the spec, the wireframe, or the rollout plan. It is the story every other artifact must obey.

What the Spine contains

- **The problem** — stated in the user's own language, with quotes from real customer conversations where possible. If you cannot describe the problem without invoking your solution, the problem is not yet understood.

- **The user** — a specific persona, a specific job, a specific workflow. Generic users produce generic products. The Spine names a person.
- **The proposed solution direction** — the shape of the answer, not the spec. The spec belongs in the Leaves; the Spine commits to the direction the Leaves must serve.
- **Success metrics** — measurable, time-bound, customer-anchored. Not vanity metrics. Not internal proxies. The numbers that, if they move, mean we did the work.
- **Scope boundaries** — what is in, what is out, what we are explicitly not doing. The strongest Spines are explicit about what they refuse, because every refusal is a commitment of focus.
- **Traceability anchor** — the matrix that maps every Leaf back to a Spine commitment. Anything in a Leaf that does not map back to the Spine is either scope creep or a missing Spine update.

What the Spine is not

The Spine is not the wireframe, the implementation plan, the rollout sequence, the engineering ticket list, or the marketing plan. Each of those is a Leaf. Treating any of them as the Spine collapses the methodology — because Leaves are written from a stakeholder's perspective, and a stakeholder's perspective is not the customer's perspective. The Spine is the customer's perspective, made explicit.

The most common Spine failure is to treat the prototype as the Spine. The prototype is a Leaf, written for engineers and designers; it inherits the medium's biases (visible elements, interactive flows, screens). The customer's problem is rarely shaped like a screen. If the prototype becomes the Spine, the team optimizes for the prototype, and the original problem quietly drifts out of the conversation.

Why customer empathy lives here

Customer empathy is not a soft skill in this methodology. It is a structural input. The Spine is empathy in document form — it is the mechanism by which the PM's customer understanding becomes the team's shared frame. Without it, the Spine is fiction, the Leaves are decorative, and the project ships into a vacuum.

Generative tools amplify whatever truth or untruth the Spine encodes. They are force multipliers for being right and for being wrong, in equal measure. A team that ships fast against a wrong Spine ships the wrong product fast. The new tools have not reduced the importance of customer truth; they have raised the cost of getting it wrong.

The Spine is the floor. Without it, nothing else in this methodology matters.

5. The Leaves: High-Resolution Artifacts

If the Spine is the idea, the Leaves are how the idea reaches different stakeholders in the language each discipline already speaks. A Leaf is a high-resolution artifact rendered in the medium that discipline natively consumes — a Mermaid diagram (of varying granularity) for architects and engineering, a coded prototype for frontend, an interactive tour for sales, a state catalogue for designers, a traceability matrix for compliance and QA. The Leaf is not the PM's translation of the discipline's job. It is the artifact the PM produces so the discipline does not have to translate prose to start working.

Each Leaf has three properties: a specific consumer, a specific format, and a specific job to do. The consumer is the function that will act on the Leaf. The format is the medium that function actually uses. The job is the translation loss this Leaf eliminates.

A note that matters more than it sounds. The Leaves do not replace each discipline's expertise. They meet experts halfway. The Architecture Leaf is not the architect's deliverable; it is the proposal the architect refines. The Engineering Leaf is not the frontend engineer's commit; it is the scaffold the engineer extends. The GTM Leaf is not the sales team's pitch; it is the demo the salesperson runs from. Each discipline retains its expertise; the methodology bridges the gap that traditionally exists between a high-level PRD that no discipline can act on and the artifact each discipline actually produces. The model is not extractive — it is collaborative. It saves discipline experts from re-deriving the artifacts they would have produced anyway, by giving them a halfway draft to refine.

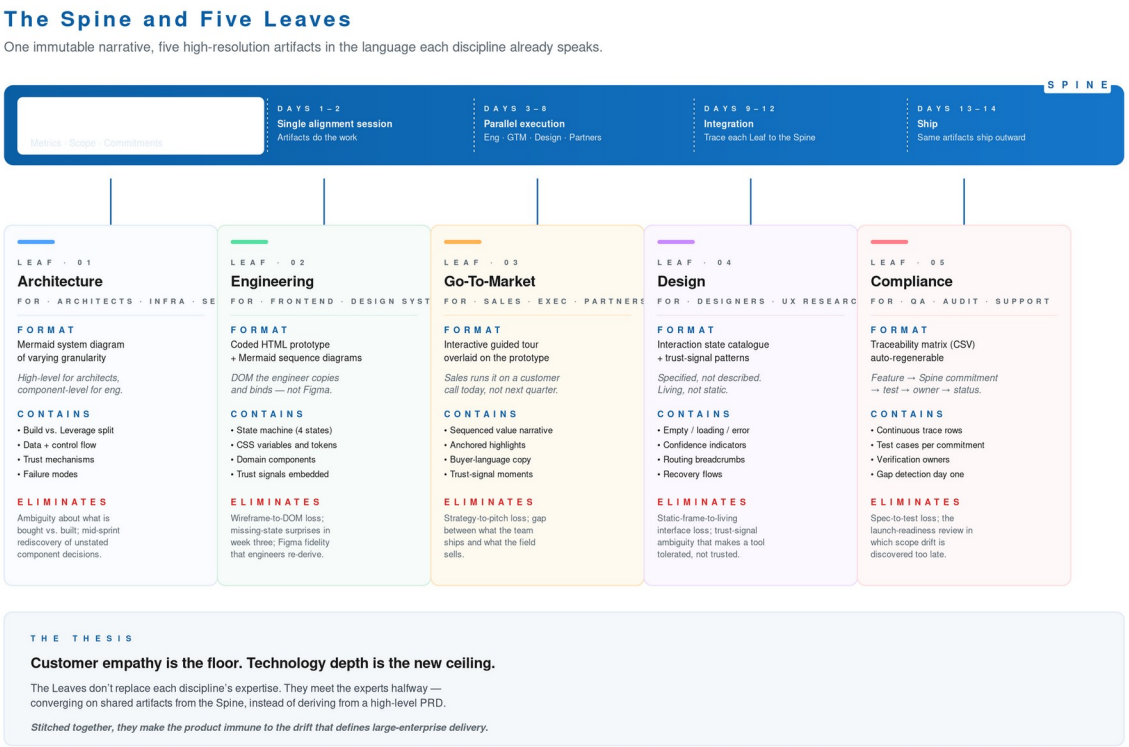

Figure 1. The Spine and Five Leaves — one immutable narrative, five high-resolution artifacts in the language each discipline already speaks.

The canonical Leaves

Five of the Leaves are disciplinary — each one is the artifact a specific function consumes. The sixth — the Sequencing Leaf — is a meta-Leaf: it doesn't serve a single discipline, it orchestrates the work that produces the others. The disciplinary five answer "what does each discipline get?" The sixth answers "in what order should we build them?"

Leaf	Consumer	Format	Translation loss eliminated
Architecture	Architects, infra, security	System diagram with explicit Build vs Leverage split	Prose-to-system-design loss; ambiguity about what is bought vs. built
Engineering	Frontend, design system, integration eng.	Working coded prototype demonstrating layout, state, and interaction	Wireframe-to-DOM loss; ambiguity about state, motion, error handling
Go-To-Market	Sales, marketing, executives, design partners	Interactive guided tour of the value proposition	Strategy-to-pitch loss; abstract concept-to-business-value loss
Design	Product designers, UX researchers	Explicit interaction states, micro-patterns,	Static-frame-to-living-interface loss; ambiguity

Leaf	Consumer	Format	Translation loss eliminated
		motion, trust signals	about dynamic states
Compliance	QA, compliance, audit, support	Traceability matrix mapping every committed feature to Spine + test	Spec-to-test loss; ambiguity about what was actually committed
Sequencing	The dev team itself — engineers, designers, marketers self-routing	Work-item graph with slice memberships, status, and dependencies — filterable by slice and discipline	The 'what should I pick up next?' Slack message; PM-as-bottleneck for daily routing

5.1 The Architecture Leaf

The Architecture Leaf is a system diagram with explicit Build versus Leverage decisions. It shows the major components, the data and control flow between them, the boundary between what the team is building and what it is composing from existing infrastructure, and the trust mechanisms that make the system safe to operate. The format is a Mermaid system diagram of varying granularity: high-level for the architect (subsystems and boundaries), component-level for engineering (module-by-module flow). The same source file renders at multiple resolutions; the audience determines which one is shown.

Why the PM produces it: because the Build versus Leverage decision is a product decision, not an engineering decision. It is the decision about where the moat lives. A PM who delegates this decision is delegating the product's strategic shape to whoever drew the diagram, which is rarely the right person to make the call. The architect refines the diagram; the PM proposes it.

What it eliminates: the recurring meeting in which the team rediscovers, mid-sprint, that two components were assumed to exist by different functions. The Architecture Leaf turns that disagreement into a one-time conversation at the start, mediated by a single shared artifact.

5.2 The Engineering Leaf

The Engineering Leaf is a working coded prototype, complemented by Mermaid sequence and component diagrams at module-level granularity. The prototype is not a screenshot, a Figma frame, or a description — it is a file the engineer can open, a DOM the engineer can inspect, a set of CSS variables the engineer can copy, a state machine the engineer can trace. It demonstrates the interactions, the loading states, the error states, the empty states, and the trust signals — at the resolution at which they will actually be implemented. The accompanying Mermaid diagrams give engineering the call graph and sequence flow that the prototype does

not surface visually. Together, the two formats meet engineering exactly where engineering already works.

Why the PM produces it: because the wireframe-to-DOM translation is the single largest source of fidelity loss in modern product development. Designers produce frames optimized for design review; engineers re-derive structure from those frames; the rederivation is approximate. The Engineering Leaf compresses this chain by producing the DOM as the artifact.

What it eliminates: the third-week realization that the design did not account for the empty state, the loading state, the long-string state, the error state, or the offline state. Because the prototype is interactive, these states are surfaced before engineering starts.

5.3 The Go-To-Market Leaf

The Go-To-Market Leaf is an interactive guided tour of the value proposition. It is not a slide deck, a one-pager, or a demo script. It is a clickable artifact that walks the viewer through the product's value, in sequence, with the actual interface elements highlighted as the narrative unfolds. Sales can run it on a customer call. Marketing can extract assets from it. Executives can click through it in five minutes and arrive at a coherent mental model of the product.

Why the PM produces it: because the strategy-to-pitch translation is the second largest source of fidelity loss. Sales receives a strategy memo and produces a pitch from it; the pitch optimizes for a different audience and frequently drifts from the strategic frame. The GTM Leaf gives sales the demo without requiring them to derive it.

What it eliminates: the gap between what the team thinks it is shipping and what the field thinks it is selling. The tour is the same artifact for both. If the field cannot sell what the tour shows, the team has shipped the wrong thing — or the tour is wrong, which is also useful information, surfaced early.

5.4 The Design Leaf

The Design Leaf documents the interaction patterns and trust signals that distinguish a credible product from a passable one. It catalogues the loading indicator behavior, the typing-state animation, the confidence indicators, the source citations, the routing breadcrumbs, the empty states, and the error recovery flows. In an AI product, the Design Leaf is where the difference between a tool the user trusts and a tool the user tolerates is encoded.

Why the PM produces it: because the trust signals are product decisions, not visual decisions. Whether a confidence score is shown, whether a source is cited, whether a routing path is visible — these are decisions about how the user is invited to verify the system. The PM owns the verification model. The designer refines the visual expression.

What it eliminates: the late-stage discovery that the product technically works but feels unreliable. Trust is built through the small interactions, and the small interactions need to be specified before the team starts arguing about whether to add them.

5.5 The Compliance Leaf

The Compliance Leaf is a traceability matrix. Every feature in the prototype maps to a Spine commitment. Every Spine commitment maps to one or more test cases. Every test case maps to a verification owner. The matrix is the document of record for what the team has actually committed to ship.

Why the PM produces it: because the matrix is the structural representation of scope. Without it, scope is whatever the prototype implies, and the prototype is whatever the team had time to build. With it, scope is what the Spine committed and what the matrix can verify.

What it eliminates: the launch-readiness review in which the team discovers that three features in the prototype were never explicitly committed, and two features in the Spine were never built. The matrix surfaces this in week one rather than week thirteen.

5.6 The Sequencing Leaf

The Sequencing Leaf is a graph view of the project, rendered from the work items themselves. Nodes are work items — stories, tasks, epics. Edges are dependencies, typed as either blocks (the upstream item must complete before the downstream item can begin) or informs (the upstream item is preferable, but not strictly required). Each work item carries an effort estimate, a user-visible flag, the discipline that owns it, the Spine commitment it traces to, the slice memberships it belongs to (MVP1, Pilot, GA, etc.), and a live status (not started / in progress / done). The format is a YAML or JSON spec for the items and edges, alongside a Mermaid graph for visual reading. The two files share IDs so the graph and the data are always consistent.

The point is not to be a strategy artifact the PM consults during planning. The point is to be a routing tool the dev team self-services from, so a fast team does not need to ping the PM every time someone wants to know what to pick up next. The PM authors and maintains the graph; the team filters by their slice and discipline, sees what's ready to start, and picks something up. The PM stops being a bottleneck for daily routing because the artifact answers it.

Backlog tools surface dependencies as flat tabs, not topologies. Jira and Linear, GitHub Projects, Notion — none give the team a graph view of what runs in parallel and what is locked into a sequential chain. Story Mapping is the closest cultural precedent; PERT and CPM contain the graph math; Microsoft Project still offers network diagrams. None of these live where the modern backlog lives. The Sequencing Leaf is the synthesis: topology of CPM, slicing of Story

Mapping, work-item data model of modern PM tooling, and the version-controllability of the other Leaves.

What the team gets: a Ready Now queue (work items whose blockers are all done, filtered to slice + discipline), slice filtering (MVP / Pilot / GA), the critical path (longest sequential chain — the schedule's true bottleneck), bandwidth-aware calendar time (given N engineers, the minimum time to ship the slice), and unstated dependencies surfaced by the act of drawing the edges. The interactive demo published with this paper renders the graph for a generic save-and-resume feature with fifteen work items and three slices — toggling click-to-cycle status, a Monday-morning Ready Now queue, and live metrics. Try it before adopting; it is the smallest practical Sequencing Leaf.

5.7 Why This Works: The Leaves Are Code

There is a structural property of the methodology that explains why it scales beyond a single cycle, and it is worth naming explicitly because it changes how the artifacts are managed. The Leaves are code.

The coded prototype is code. The architecture diagram, expressed as Mermaid or as a structured spec, is code. The interactive tour is code. The traceability matrix is a structured file under version control. Even the Spine, when written with discipline, is text in a repository — not a slide, not a Notion page that drifts, not a Google Doc with seventeen comment threads. Every Leaf is a file.

This matters because it means a change in one place can be reflected everywhere. When the Spine updates — a metric is refined, a scope item is removed, a user persona is sharpened — the prototype gets updated, the tour gets re-shot, the architecture diagram gets adjusted, and the traceability matrix gets regenerated. The artifacts are not parallel documents that drift; they are derivatives of the same source. A single edit propagates to every team's resolution of the story.

This is not a small property. It is the difference between a methodology that holds together for one cycle and one that compounds across many. In the document-and-deck era, every artifact was a one-off — a strategy memo, a slide deck, a Figma frame, a Confluence page. Updating the strategy meant updating five things by hand, and three of them were always out of date by the next standup. In the Spine-and-Leaf era, the artifacts share a substrate. Updating the Spine is a real edit, not a coordination problem. The team's resolution of the change happens by re-running the artifacts, not by scheduling a sequence of cross-functional meetings.

It also means the methodology is engineerable. The Leaves can be templated. The traceability matrix can be auto-generated from the prototype's component tree. The tour script can be regenerated from the prototype's state catalogue. As the team runs more cycles, the artifacts

converge into a system, and the system carries the methodology forward whether or not the original PMs are still in the room. The methodology does not depend on heroics; it depends on infrastructure.

The Leaves are code. That is not a metaphor. It is the operational reason this delivers value.

5.8 What Happens When the Spine Changes

The single most under-discussed property of any methodology is what it does when the core truth changes. The Spine is the immutable referent in this model — but immutable does not mean unchangeable. It means that when it changes, the change is deliberate and propagated. The methodology's value is not that the Spine never moves; it is that when the Spine moves, the rest of the team's resolution of the product moves with it, in a known sequence, with named owners notified.

When the Spine changes, four things happen, in this order:

- **Step 1 — version + log.** The Spine version is incremented (for example, spine_version: 0.1 → 0.2). A one-line entry is added at the top of the file: the date, what changed, and the reason. This is the audit trail; without it, propagation is invisible.
- **Step 2 — Architecture Leaf review.** Walk the system diagram for any boundary that no longer holds — a Build/Leverage decision, a trust mechanism, a data flow. The architect is notified. If the change is structural, the Leaf is updated and re-circulated.
- **Step 3 — Engineering and Design Leaves review.** Walk the prototype for any state, copy, or interaction that drifts from the new direction. Frontend, design system, and design leads are notified. The state catalogue is updated; the prototype is rebuilt; trust signals are reviewed.
- **Step 4 — GTM Leaf review and Compliance regeneration.** The tour script is reviewed for any value claim that no longer reflects what is being built. Sales and marketing are notified. The Compliance Leaf — the traceability matrix — is regenerated automatically from the prototype's component tree. It is the Leaf that catches the rest if a propagation step is missed.

The propagation is not optional and not slow. Because the Leaves are code (Section 5.7), regeneration is closer to a build step than a meeting. The role of the PM at this moment is to write the Spine update, run the propagation, and confirm the four notifications. That is the work the model produces value from — not a meeting that announces a change, but a deliberate update of the source of truth that all artifacts derive from. The methodology suggests, in code, who to inform and what to update so the entire project keeps a single source of truth.

Stitched together, the five Leaves and the propagation discipline make the product immune to the drift that defines large-enterprise delivery — the drift produced when every discipline has its own pull or push, its own private artifact, its own version of the customer. In a Spine and Leaf

system, every discipline has its own Leaf. None has its own Spine. That is what one source of truth means in practice.

6. Build vs. Leverage: The Strategic Layer

The single most important question in AI-era product delivery is: where is the moat? Foundation models, vector stores, evaluation frameworks, and integration APIs are now commodity. The team that uses them is not differentiated by using them; the team is differentiated by what it composes on top of them. The Build versus Leverage decision is the product's strategic shape, expressed as a system.

The decision sits in the Architecture Leaf because that is where it is most visible. But it is a product decision, owned by the PM, and it requires the technology depth to make correctly. A PM without depth defaults to one of two failure modes: build everything, or leverage everything. Both are wrong, in opposite directions.

LEVERAGE — commodity / infrastructure	BUILD — the moat
Foundation models for general reasoning, summarization, classification.	Domain-specific orchestration logic — the workflow shape that reflects how this user actually works.
Vector databases and embedding pipelines for retrieval.	Trust mechanisms calibrated to the user's risk tolerance — the deterministic-to-autonomous spectrum, the verification UI.
Standard integration APIs for ITSM, CRM, identity, telemetry.	Bespoke visualizations that earn the user's understanding of complex domain state.
Off-the-shelf evaluation infrastructure (scoring, observability, tracing).	Domain evaluation frameworks that encode what a correct answer looks like in this user's world.
Common UI primitives, design systems, accessibility libraries.	The interaction model that distinguishes this product from a generic chat surface.

Heuristic

If a competitor can also use it tomorrow, leverage it. If it encodes how your user thinks, build it.

The phrase "how your user thinks" is doing a lot of work in that heuristic. It refers to the mental model the user brings to their work — the categories they use, the sequences they expect, the failures they fear, the verifications they perform. A foundation model does not know any of this. The product's job is to encode it. That encoding is the Build.

The mistake of building everything is the mistake of treating commodity infrastructure as differentiation. The team that builds its own foundation model in 2026 is making a category error: the moat is not in the model, it is in what the model is composed with. The mistake of leveraging everything is the opposite category error: the team ships fast, and ships the same product as its competitors, because the differentiation was supposed to live in the layer they declined to build.

The right answer is almost always a small Build sitting on top of a large Leverage. The Build is the 5–10% of the system that encodes the user's domain. The Leverage is everything else. The Architecture Leaf is where this proportion is made visible, and it is what gives the executive sponsor the confidence to fund the work — because the system diagram makes the moat legible.

7. The Operating Rhythm

The Intelligence Supercycle is shrinking the window between idea and ship. The teams that win the next two years will not be the ones with the best four-week plan; they will be the ones who can run a credible two-week cycle inside a market that increasingly rewards weekly delivery. The methodology in this paper compresses the alignment phase that used to dominate the first half of every sprint. The cost of that compression is paid by the PM, in the form of the Spine and the first drafts of the Leaves, before the sprint begins. The benefit is that engineering, design, and go-to-market can run in parallel from day one rather than serially from day eight — and that parallelism is what lets the team ship inside the supercycle's compressed window rather than chasing it.

Window	Activity	What's different from the old rhythm
Day 0	PM produces Spine + first draft of Architecture, Engineering, and GTM Leaves.	In the old rhythm, this was the PRD-only phase, and it took two weeks. The Leaves are produced in parallel with the Spine, not after.
Days 1–2	Alignment. Team reviews the Spine and Leaves together in a single session.	In the old rhythm, alignment was a sequence of one-on-ones. Now the artifacts do most of the alignment work; the meeting confirms.
Days 3–8	Parallel execution. Frontend wires the Engineering Leaf to real state. Backend builds the orchestration logic from the Architecture Leaf. GTM tests the tour with design partners.	In the old rhythm, frontend waited on design, design waited on PRD review, GTM waited on a working build. Now all three start on day three.
Days 9–12	Integration. Each Leaf is reconciled against the Spine via the Compliance Leaf	In the old rhythm, integration discovered scope drift late. The matrix surfaces drift continuously.

Window	Activity	What's different from the old rhythm
	(traceability matrix).	
Days 13–14	Ship. The same artifacts that aligned the team now align the customer.	In the old rhythm, launch assets were produced after the build. Now they are the build's twin, written from day zero.

The two-week cycle is not a magic number. The rhythm works at four-week or eight-week scale as well. What matters is the inversion: the alignment cost is paid up front, by the PM, in artifacts that aren't documents, and the back half of the cycle is freed for execution rather than translation.

8. Benefits at a Glance

The methodology earns its keep across multiple vectors. The summary below is the operational case — what changes when teams shift from document-and-deck grammar to Spine and Leaf grammar. Each row pairs to material elsewhere in the paper; the table is the index, not the argument.

Vector	What changes
Time to alignment	Hours, not weeks. Artifacts do the alignment work; the meeting confirms.
Velocity	Teams ship inside the Supercycle's compressed window. Field cases: 10× faster artifact creation, 33% lifts in deal velocity, 28% reductions in CAC.
Drift immunity	Every discipline has its own Leaf; none has its own Spine. Silos Gartner names as "competing trios of insight" collapse into a shared trio.
Translation fidelity	Each discipline receives the artifact in the language it consumes. No prose-to-image-to-code chain.
Pivot + compliance speed	Spine change propagates mechanically (Section 5.8). Traceability regenerates from the prototype. Scope drift surfaces continuously, not at week thirteen.
Onboarding + retention	New team members read the Spine + one or two Leaves and are current. Engineers and designers refine halfway artifacts rather than re-deriving from PRDs.

The practice is where the value is. The principles are easy to nod at; the practice — actually producing the Leaves, actually propagating Spine changes, actually meeting each discipline halfway — is where the team learns whether the methodology fits the work. Teams that run it for three or more cycles describe the same thing: they will not go back. The translation chain feels expensive once you have stopped paying for it.

9. Customer Empathy: Still The Foundation

Nothing in this methodology reduces the importance of customer empathy. If anything, it raises it. The Spine encodes the customer's truth into a single artifact, and every Leaf is a high-fidelity expression of that truth. If the truth is wrong, the methodology amplifies the error — faster, more visibly, and at higher resolution than the old methodology ever could.

The empathy work is therefore the highest-leverage hour the PM spends. A poorly understood user produces a poorly written Spine, and a poorly written Spine produces Leaves that are technically excellent and strategically wrong. The team will execute beautifully against the wrong target, and the new tools will let them do it faster than ever.

Empathy in this methodology is not an inspirational stance. It is a structural input. It looks like real customer interviews with real customers, transcripts that get re-read, observed workflows that get re-observed, pain quotes that find their way into the Spine in the user's own words. It looks like a PM who can describe the user's day in concrete, sequenced detail without referring to a slide.

The risk is that the new tools make the empathy work feel optional. They don't. They make it more consequential. A team that ships fast against a wrong Spine ships the wrong product fast — and the new tools have collapsed the cost of shipping fast, which means the cost of being wrong has correspondingly risen. The empathy work is the only thing that distinguishes a fast-shipping team from a fast-shipping team that ships the right product.

Empathy is the floor. Without it, none of the speed matters.

10. Technology Depth: The New Ceiling

The PM who can read a system diagram, propose a Build versus Leverage split, sketch a data flow, and prototype an interaction has a different career trajectory than the PM who cannot — because the first is operating with tools the second hasn't learned to use. Depth doesn't mean becoming an engineer. It means understanding the cost curves, failure modes, and leverage points of the technology stack well enough to make the right tradeoffs before engineering is staffed: that a foundation model call costs less than a database query, that a vector retrieval is fast but a re-ranking step is slow, that an evaluation suite without adversarial cases lies.

These are not edge concerns. They are the strategic shape of the product. A PM without depth defaults to whatever the loudest engineer says — sometimes right, frequently optimized for the engineer's convenience rather than the user's experience. A PM with depth proposes the architecture and invites the engineer to refine it. The conversation is between peers.

Empathy as floor and depth as ceiling is not a balance — it is a stack. Empathy comes first; without it, depth produces clever solutions to wrong problems. Depth comes second; without it,

empathy produces correct problems with timid solutions. Together they produce the PM who leads delivery in this era. The constraints around the role have lifted. The opportunity is to do the work the discipline always wanted, with the tools that finally make it possible.

11. Anti-Patterns

Every methodology has predictable failure modes. The Spine and Leaf model has five, each recognizable by the symptom it produces.

- **PRD-and-pray.** Symptom: PM writes a 30-page doc and waits six weeks. Cause: no Leaves. Fix: produce the Engineering Leaf alongside the Spine in week one, even if rough — its job is to externalize the model so the team can disagree with something concrete.
- **Mock without build.** Symptom: beautiful Figma frames re-derived in code with loss of fidelity. Cause: the design artifact is optimized for review, not implementation. Fix: produce the Engineering Leaf as a coded prototype, not a frame.
- **Build without Spine.** Symptom: a working prototype exists; the team optimizes it without a problem statement; the prototype quietly becomes the Spine. Cause: Engineering Leaf produced before the Spine. Fix: the Spine must precede the Leaves. The most common failure in this model, and the most damaging.
- **Leverage everything.** Symptom: ships fast, has no moat; a competitor ships the same product next quarter. Fix: the Architecture Leaf must identify a Build, even a small one. A small deliberate Build is differentiation; no Build is a feature.
- **Build everything.** Symptom: ships slowly, runs out of runway. Fix: if a competitor can use it tomorrow, leverage it. The team's job is not to build technology; it is to ship the system that encodes the user's domain.

12. How To Adopt: Starting Tomorrow

The methodology does not require permission, a reorganization, or a tool migration. It requires the PM to start producing different artifacts. The adoption path below is the one I recommend for teams that have not run this model before.

- **Step 1 — Pick a small in-flight project.** Not the largest one. Not the most political one. Choose a project where the Spine is reasonably clear and the team is willing to try a different sequence.
- **Step 2 — Write the Spine in plain language.** Two to four pages. Problem, user, direction, success metrics, scope, what we are not doing. Circulate it. Solicit pushback. Update it once. Then freeze it.

- **Step 3 — Add one Leaf this sprint.** For most teams, the Engineering Leaf is the right place to start, because it is the artifact whose absence is felt most acutely. A coded prototype, even rough, will reset every conversation with engineering.
- **Step 4 — Add a second Leaf next sprint.** Usually the GTM Leaf or the Architecture Leaf. The choice depends on which audience is most under-served by the current process. If sales is repeatedly mis-pitching the product, start with GTM. If engineering is repeatedly rediscovering scope, start with Architecture.
- **Step 5 — Add the Compliance Leaf in cycle three.** The traceability matrix is most useful once the team has produced enough other Leaves to need traceability. It will feel premature in cycle one and overdue in cycle four. Aim for cycle three.
- **Step 6 — Audit after three cycles.** Compare the time-to-aligned-team in the new model against your historical baseline. The improvement is usually large enough that the team will not let you go back. If it is not, the Spine is probably weak; revisit step two.

The methodology compounds. Each cycle produces artifacts that become reference material for the next cycle. After a year, the team has a library of Leaves that serve as patterns, and new projects start with a Spine and a set of templated Leaves rather than with a blank page. This is the long-run benefit: the methodology converts each project into infrastructure for the next one.

13. Closing: Rediscovery

The Product Manager role was not broken. The constraints around it changed. For most of the discipline's history, producing a high-fidelity artifact was expensive — so the PM had no choice but to communicate in prose. The role, the rituals, and the failure modes all evolved around that constraint. The constraint has now lifted. The opportunity is not to use the new tools to write better PRDs. The opportunity is to retire the PRD as the primary artifact and replace it with a Spine and a set of Leaves that each stakeholder can use directly, without translation.

This is not a transformation of the role. It is a rediscovery of what the role was always trying to be. Customer empathy was always the foundation. Technology depth — exercised through the production of high-resolution artifacts — is the new ceiling. The methodology in this paper is how those two are combined into a discipline that delivers products in less time, with less drift, and with more customer truth in them.

A note on accountability

One final point. The new tools are not failing to deliver value — they are delivering exactly what we ask of them. If a team uses an AI assistant to write a faster PRD, they get a faster PRD. The tool did its job. The methodology did not move. The disappointment organizations express about generative tooling is, in most cases, an honest description of the tooling's failure to compensate for an unchanged process. The tools cannot do that. Only we can. The methodology is what we choose to ask the tools to produce.

The question to ask, every time a new tool lands on the desk, is not "can I produce my old artifacts faster?" but "can I produce a different artifact that obviates one of my old workflows?" That question is the methodology, in compressed form.

One day, AGI will do everything. Until that day, the ecosystem is generous; the baseline is on us. Every Leaf is a baseline raised. Every refusal to raise it is a baseline ceded.

The wave is here. The methodology is the surfboard. The PM who picks it up will deliver in this era. The PM who does not will be coordinating, while someone else builds.

Appendix A: Starter Templates

These templates are the smallest workable starting point for each Leaf. They are deliberately generic — copy them, replace the angle-bracket placeholders with your project's specifics, and run them. The first version of every artifact should exist within an hour of starting; the templates exist to make that one-hour goal possible. Place all five files in a single repository so the artifacts share a substrate (the property described in Section 5.7) and a change to the Spine propagates through the rest.

A short note for aspiring PMs: if you do not yet own the project, run these templates at the scale you do own — a feature, a workflow, a single screen. The methodology compounds at any scale. The hard part is not authoring the templates; it is committing to update them when the world changes. Do that, and the artifacts become trustworthy. Do not, and they become decoration.

A.1 The Spine — spine.md

Plain Markdown, two to four pages when filled in. Frontmatter for status and version. Body sections for problem, user, direction, success metrics, scope, and traceable commitments. Every Leaf must trace back to a numbered commitment in the final section.

```
---
title: <project name>
status: draft          # draft | aligned | shipped
owner: <PM name>
spine_version: 0.1
last_updated: 2026-XX-XX
---

# Problem
<In the user's own words. Use a real quote if you have one.
 What is the user trying to do, and what makes it expensive today?>

# User
<A specific persona. Name them. Describe their day, their tools,
 their pain points. Generic users produce generic products.>

# Proposed solution direction
<The shape of the answer, not the spec. One paragraph.
 Constraint: should be readable in 90 seconds.>

# Success metrics
<Measurable, time-bound, customer-anchored. 2–4 metrics maximum.>
- M1: <metric>
- M2: <metric>

# Scope
**In:**  <what we are committing to ship>
**Out:** <what we are explicitly not doing>
```

```
# Commitments (traceability anchor)
Every Leaf must trace back to a numbered commitment here.
- C1: <commitment>
- C2: <commitment>
- C3: <commitment>
```

Common mistakes: writing the Problem in the team's language; smuggling the solution into the Problem; omitting the Out-of-scope list; numbering more than seven commitments.

A.2 The Architecture Leaf — architecture.mmd (Mermaid)

Single Mermaid file. Two subgraphs — BUILD and LEVERAGE — make the strategic split visible. Renders in any Markdown viewer that supports Mermaid (GitHub, VS Code, Obsidian, modern docs platforms). The diagram is the proposal; the architect refines it.

```
graph TD
   User[<User Persona>]
   UI[UI Surface]
   User -->|Intent| UI

   subgraph "BUILD — Your Moat"
      UI --> Router[Domain Orchestrator]
      Router -->|Workflow A| WorkflowA[<Domain Workflow>]
      Router -->|Workflow B| WorkflowB[<Domain Workflow>]
      Router --> Trust[Trust + Verification UI]
   end

   subgraph "LEVERAGE — Commodity"
      WorkflowA <-->|Reasoning| LLM[Foundation LLM]
      WorkflowB <-->|Retrieval| RAG[Vector Store + RAG]
      Router    <-->|Integration| External[Standard APIs]
      Router    <-->|Tracing| Eval[Evaluation Infra]
   end

   classDef build    fill:#0B5FA5,stroke:#fff,color:#fff;
   classDef leverage fill:#5C6770,stroke:#fff,color:#fff;
   class Router,WorkflowA,WorkflowB,Trust,UI build;
   class LLM,RAG,External,Eval leverage;
```

Common mistakes: putting infrastructure inside BUILD; not labeling edges (the value is in the data flow); leaving Trust as an afterthought.

A.3 The Engineering Leaf — index.html (scaffold)

A self-contained HTML page with CSS variables, a state container, and explicit slots for empty / loading / error / success states. Open it in a browser; the engineer can copy the DOM and bind it to real state. This scaffold is the smallest version that still demonstrates the four states every interface needs.

```
<!doctype html>
```

```
<html lang="en">
<head>
<meta charset="utf-8" />
<title><Project Name></title>
<style>
  :root {
    --bg: #ffffff; --fg: #1a1a1a; --muted: #6b7280;
    --accent: #2563eb; --good: #059669;
    --warn: #d97706; --error: #dc2626;
    --rule: #e5e7eb; --surface: #f9fafb;
    --font-body: system-ui, -apple-system, sans-serif;
    --font-mono: ui-monospace, SFMono-Regular, monospace;
  }
  body { margin: 0; font-family: var(--font-body);
    color: var(--fg); background: var(--bg); }
  .app { display: grid; grid-template-rows: auto 1fr auto;
    min-height: 100vh; }
  header { padding: 1rem 1.5rem;
    border-bottom: 1px solid var(--rule); }
  main { padding: 1.5rem; }
  .state-empty, .state-loading, .state-error {
    padding: 2rem; text-align: center; color: var(--muted); }
  .state-loading { color: var(--accent); }
  .state-error { color: var(--error); }
  .confidence { display: inline-flex; padding: 2px 8px;
    border-radius: 999px; font-family: var(--font-mono);
    font-size: 12px; background: var(--surface); }
</style>
</head>
<body>
  <div class="app">
    <header><h1><Product Name></h1></header>
    <main id="root">
      <!-- Replace with your domain components.
           Each component should have explicit states:
           empty, loading, error, success. -->
      <div class="state-empty"   id="empty">Type a query to begin.</div>
      <div class="state-loading" id="loading" hidden>
        Working… <span class="confidence">routing</span></div>
      <div class="state-error"   id="error"   hidden>
        Something went wrong.</div>
    </main>
    <footer></footer>
  </div>
  <script>
    // State machine: empty → loading → success | error
    // Replace with your real workflow logic.
  </script>
</body>
</html>
```

Common mistakes: skipping the empty state; loading without a confidence signal; designing success without designing error.

A.4 The Go-To-Market Leaf — tour.yml

A YAML tour script. Each step has an anchor (the DOM element to highlight), a headline (one phrase that names the value), a body (one or two buyer-language sentences), and a Spine

commitment ID so the tour traces back to a real promise. Sales runs the tour on the prototype; marketing extracts copy from it.

```
# tour.yml — Spine and Leaf GTM template
tour: <Product> Value Tour
version: 0.1
spine_version: 0.1   # must match the Spine you traced from

steps:
  - step: 1
    anchor: "#hero"
    headline: <One phrase that names the value>
    body: >
      <One or two sentences. Speak the buyer's language,
       not the engineer's.>
    spine_commitment: C1

  - step: 2
    anchor: "#workflow-a"
    headline: <Demonstrate a concrete pain solved>
    body: <The before-state, then the after-state. Specific.>
    spine_commitment: C2

  - step: 3
    anchor: "#trust-signals"
    headline: <Show how the user verifies the system>
    body: <Confidence indicators, source citations, override path.>
    spine_commitment: C3

  - step: 4
    anchor: "#empty-state"
    headline: <What the user sees on a quiet day>
    body: <Quiet days are 80% of the user's life. Design for them.>
    spine_commitment: C1
```

Common mistakes: team vocabulary instead of buyer language; leading with features, not value; skipping the trust-signal step; writing more than seven steps.

A.5 The Compliance Leaf — traceability.csv

A CSV that maps every committed feature to a Spine commitment and a test case. Update it continuously, not at launch. The matrix is the document of record for what you have actually committed to ship; if a feature is in the prototype but not in the matrix, it is either out of scope or the matrix is missing a row — both worth a conversation.

```
feature_id,feature_name,spine_commitment,test_case,verification_owner,status
F1,Domain workflow A,C1,User completes <task> in under <X> seconds,QA Lead,not_started
F2,Confidence indicator,C3,Every output displays a confidence score,Frontend Eng,not_started
F3,Source citation,C3,Every claim links to a source document,Backend Eng,not_started
F4,Empty state,C1,Empty state copy + CTA renders for first-time users,Design,not_started
F5,Error recovery,C2,Retry path appears within 2s of a failed call,Frontend Eng,not_started
```

Common mistakes: starting the matrix at week thirteen; leaving the spine_commitment column blank; marking complete before verification runs.

A.6 The Sequencing Leaf — sequencing.yml + sequencing.mmd

Two files. The YAML carries the data — nodes, edges, slices. The Mermaid carries the visualization. They share IDs, so a change in one is reflected in the other. The Mermaid diagram renders in any GitHub or VS Code preview. The YAML is what your tooling reads when computing critical paths and bandwidth-aware schedules.

The two-file split matters: the YAML is the substrate the rest of the methodology depends on (the Compliance Leaf can join its features against this, for example), and the Mermaid is the rendering humans look at. Either can be regenerated from the other when the tooling is in place; for now, keep them hand-synced.

sequencing.yml

```yaml
# sequencing.yml — Spine and Leaf · Sequencing template
# Work-item-centric. Slice memberships and status live on each item.
spine_version: 0.1

slices:
  - id: mvp
    name: Thin MVP
    description: Local-only save & resume. Ships in days.
  - id: pilot
    name: Pilot
    description: Server-backed with login + GTM prep.
  - id: ga
    name: GA
    description: Full feature.

items:
  - id: N1
    title: <story title — short imperative phrase>
    discipline: frontend   # frontend | backend | fullstack | design | gtm | data | compliance | docs
    effort: 1              # ideal days; supports decimals
    user_visible: true     # does the customer see this when shipped?
    spine_commitment: C1   # which Spine commitment this serves
    slices: [mvp, ga]      # which slices this item belongs to
    status: not_started    # not_started | in_progress | done

  - id: N2
    title: <story title>
    discipline: backend
    effort: 3
    user_visible: false
    spine_commitment: C1
    slices: [pilot, ga]
    status: not_started

edges:
```

```
  - from: N1
    to: N6
    type: blocks          # blocks | informs
    reason: <why this dependency exists — one phrase>

  - from: N2
    to: N6
    type: informs         # N6 prefers N2 but works with N1
    reason: <...>
```

sequencing.mmd (Mermaid)

```
graph LR
    %% Disciplinary color classes
    classDef frontend fill:#4DA6E8,stroke:#fff,color:#fff;
    classDef backend  fill:#4DD88A,stroke:#fff,color:#fff;
    classDef fullstack fill:#A87CE8,stroke:#fff,color:#fff;
    classDef gtm      fill:#E8A034,stroke:#fff,color:#fff;
    classDef design   fill:#E85454,stroke:#fff,color:#fff;
    classDef compliance fill:#FF5470,stroke:#fff,color:#fff;

    %% Nodes  (id["label · effort · user_visible flag"])
    N1["N1 · localStorage · 1d · ◉"]:::frontend
    N2["N2 · backend persistence · 3d"]:::backend
    N5["N5 · authentication · 4d · ◉"]:::fullstack
    N6["N6 · resume button · 1d · ◉"]:::frontend
    %% ... add the rest from sequencing.yml

    %% Edges  (-->|type| for blocks, -.->|informs| for informs)
    N1 -->|blocks| N6
    N2 -.->|informs| N6
    N5 -->|blocks| N14
    %% ...
```

Common mistakes: too much in one graph (one Leaf per epic, not per product); forgetting to mark items user_visible; confusing blocks with informs; stale status fields; calendar days instead of ideal days; treating the graph as a PM-only artifact (the value lands when the team self-routes from it).

An interactive demo of this exact template — fifteen work items, three slices, slice + discipline filters, click-to-cycle status, a live Ready Now queue — accompanies this paper. It is the smallest practical Sequencing Leaf and demonstrates the self-service routing model on a generic SaaS feature.

Using the templates as a system

Drop all five files into a single repository — call it whatever your project is called. Each template references the Spine version it traces from. When the Spine updates (a metric refines, a scope item drops, a persona sharpens), increment spine_version, then walk through the four other Leaves and update them to match. The artifacts are now derivatives of one source rather than

parallel documents that drift. This is the Section 5.7 property made operational, and it is the difference between a methodology that holds together for one cycle and one that compounds across many.

If you adopt only one template this sprint, adopt the Spine. The other four are higher-resolution. The Spine is the load-bearing one. Without it, the rest decorate.

End of paper.